Campus Supply Desk - A2

Mailles Machiana / COMP3415 / November 8, 2026

Repo: https://github.com/McMerger/COMP3415-A2-Campus-Supply-Desk

1. System understanding

Campus Supply Desk is a terminal program for ordering campus merchandise. It can show stock, give a quote, place an order, show past orders and cancel one. Prices are whole cents. Orders and stock are saved locally in JSON; there are no real payments, taxes or shipping services.

The main parts are:

- `__main__.py`: starts the program by calling `cli.main`.
- `cli.py`: reads command arguments, converts quantities and returns JSON.
- `menu.py`: asks questions, displays dollar amounts and gets confirmation.
- `service.py`: contains `OrderService`, which checks requests and handles prices, stock, order IDs and cancellation.
- `storage.py`: contains `JsonStore`, which loads and saves state.
- `catalog.json`: supplies the five starting products when a save doesn't exist.

One flow: placing an order

In the menu, option 3 calls `read_request`. It collects the customer, products, quantities and fulfillment. The menu calls `service.quote` to show the amount. If the user confirms with y, it calls `place_order`.

Placement loads state again and checks the complete request. A previous quote doesn't reserve anything. The service builds the price and item snapshot, gives the order its next ID, deducts stock and appends the order. `JsonStore.save` writes everything to the selected file. Reopening that file brings back both history and the remaining stock.

The hardest part to maintain

The original `service.py` needed the most attention. Quote and placement copied nearly the same validation and calculation code. Placement then mixed those steps with inventory and saving. That made it harder to see whether a pricing change had reached both routes or whether a failed request could leave a partial order. The functional spec was the reference for expected behavior, rather than assuming every starter result was right.

This report covers the work after Checkpoint A. Its original submitted document was not available to compare, so this report doesn't recreate that independent submission.

2. Testing strategy and results

Tests use temporary save files. Expected amounts come from the specification, not from the same helper being tested. Here are the four required categories.

Normal behavior

Two notebooks for a regular customer should total 1000 cents with free pickup. The starter and final versions both return that result. The supplied smoke test checks each amount, not just whether the call finishes. This protects the basic buying path.

Boundary behavior

Two bags cost exactly 5000 cents. A student should get 500 cents off, but the starter returned zero. The added test failed for quote and place. Both now return 500. Another test checks delivery at 9500 and 10000 cents after discount: expected fees are 600 and zero, and both pass. These boundaries matter because a comparison like > instead of >= changes the price at the exact cutoff.

Invalid or exceptional behavior

Empty requests, unknown products, booleans, numeric strings, fractional quantities and quantities outside 1–99 should raise ValueError at the service boundary. Rejected placements must leave saved data and the next ID alone. The final tests check those outcomes, including an invalid later line. Duplicate notebook lines of 11 + 10 should fail against stock 20; the starter wrongly accepted them.

Behavior to preserve

Quotes must not create saves or reserve stock. Quote and order items/prices must agree, IDs must stay sequential, and reopening must keep stock and history. `test_lifecycle.py` checks these results, including original prices after a catalog price change. These are important because moving calculations and stock updates could otherwise change saved behavior without an obvious pricing error.

The runs

Logs 01–06 in `evidence/` show 4 starter tests passing; 27 baseline tests with 6 failing assertions; the same 6 after the early refactor; then all 27 passing after fixes. Adding CR-01 tests gave 49 tests with 5 failures and 8 errors before express support; all 49 passed afterward. The final suite now has 50 passing methods.

Log 07 holds the final run. Logs 10 and 11 rerun the same 49 tests before and after the later money-name refactor. Real command processes and scripted menu input are tested too. Seven selected wrong changes are caught in temporary copies; their output is in `08-targeted-mutations.json`.

3. Five diagnoses of the original code

References below use `service.py` at starter commit `82ed75e`, except diagnosis 4. There are three code smells, one testing problem and one additional defect.

1 - Duplicated code

Location: `OrderService.quote` and `place_order`, lines 15-60 and 65-112. Both repeat input checks, item construction, discounts and delivery calculations. A rule can be fixed in one copy and missed in the other. Move that shared policy into one helper, leaving placement to handle its extra state changes.

2 - Magic numbers

Location: the discount and delivery blocks, lines 48-52 and 98-102. The business values 5000, 10, 10000 and 600 are embedded in the calculations. Someone changing a fee has to work out which number means which rule. Named constants for the thresholds, discount percentage and delivery fee make those choices easier to find. The arithmetic 50/100 in half-up rounding is a separate calculation detail, not another delivery charge.

3 - Names don't show monetary units

Location: local `subtotal`, `discount`, `delivery` and `line_total` variables in both pricing methods, especially lines 38-60 and 88-112. The output keys say cents, but the local names don't. In a program whose menu shows dollars while its service uses cents, that leaves an avoidable unit ambiguity when editing calculations. Rename them with a `_cents` suffix. This is about making the units clear; it is separate from naming the business rules or removing repeated code. It isn't evidence of a wrong calculation by itself.

4 - Too little regression coverage

Location: the original `tests/test_smoke.py`, which has four test methods. They cover two normal quotes, reopening one order and cancelling once. They miss the exact discount threshold, combined duplicate quantities, invalid-request non-mutation and repeated cancellation. All four pass while those defects remain. Add assertions for normal, boundary, invalid and preservation cases, with real saved-data checks where relevant.

5 - Repeated cancellation changes stock again

Location: `cancel_order`, lines 133-148. There is no status check before restoring items. Starting with 8 mugs, placing two and cancelling twice gives 10 in stock. The second cancellation should leave 8. Return the existing cancelled order before touching stock or saving again. This is a behavior defect, so its correction is kept separate from refactoring.

4. Three refactorings and their checks

A – Remove the copied pricing path (diagnosis 1)

Before: quote and place each had a full copy of the request checks and price rules. After: both call `_build_quote` with the caller's loaded state. Placement adds the ID, status, stock changes and save only after that helper succeeds.

This fits the problem because there is one place to maintain the shared policy. It also avoids calling public quote from placement and loading a second state inside the same operation. `_find_order` and `_change_stock` remove smaller repeated lookup/update loops as part of that cleanup.

Verification: commit `466ab2b` and logs 02/03 show the same 27 tests and six failing assertions before and after this structural change. That is evidence for the cases exercised, not a claim that the starter was already correct. Later quote/order agreement, stock recheck and history tests pass in the final suite.

B – Give the business values names (diagnosis 2)

Before: both pricing methods embedded the discount threshold and delivery fee. After: constants such as `STUDENT_DISCOUNT_THRESHOLD_CENTS`, `FREE_DELIVERY_THRESHOLD_CENTS` and `STANDARD_DELIVERY_CENTS` state their purpose. The percentage is named too. CR-01 later adds `EXPRESS_DELIVERY_CENTS`.

This keeps the rule values visible without changing their amounts. The early constant extraction is also in `466ab2b`. Logs 02/03 retain the same results; pricing tests cover the ordinary fees and both thresholds. Fixing `>` to `>=` is a later bug fix, not part of the claim that naming constants preserved behavior.

C – Put cents in the money variable names (diagnosis 3)

Before: local amount names didn't show their units. After: they are `subtotal_cents`, `discount_cents`, `delivery_cents` and `line_total_cents`. The formulas, fulfillment strings and returned JSON keys stay the same.

This makes it easier to follow where dollar-formatted menu output meets integer service values. Commit `319fa35` contains this rename separately from new tests. Logs 10/11 run the same 49 tests against the prior service snapshot and the renamed version; both pass. This refactoring was added during the final handout review, after CR-01, rather than being presented as an earlier change.

Defect fixes are different

The $50 comparison, combined duplicate-stock check and repeat-cancellation guard intentionally correct behavior that disagreed with the spec. Commit `1cd95b9` and log 04 record those fixes. They are not counted as the three refactorings.

5. CR-01: express delivery

A - What was predicted before implementation

The preserved pre-implementation chat note said: "CR-01 is available now: it adds express delivery at a fixed $12 fee through the service, commands, and menu. I'll include tests for its pricing, persistence, cancellation, and discoverability."

That named the three affected interfaces and the intended tests. In this repo, they map to `service.py`, `cli.py` and `menu.py`; that filename mapping is explained here, not quoted as an older note. The test commit `c3e234b` came before the implementation commit `e6abcca`. `docs/cr01-notes.md` preserves the note and dates. There wasn't a separate full Part A worksheet, so this is the available planning record rather than a newly invented pre-change document.

B - Implementation and results

The service accepts express and always adds 1200 cents after the merchandise discount. It never uses standard delivery's free-delivery threshold. The command parser accepts express for quote/place, and the menu prompt lists it. The README, functional spec and command help were updated. Regular customer and pickup remain the defaults.

The client examples pass: two notebooks for a regular customer total $22; two hoodies for a student total $84; three hoodies cost $120 with standard delivery and $132 with express. Quotes and orders agree. Saved express orders keep their items, fulfillment and prices through retrieval, reopening and cancellation. Stock comes back once and IDs aren't reused. Unsupported fulfillment still fails.

C - What actually changed

`_build_quote` in `service.py`, `build_parser` in `cli.py` and `read_request` in `menu.py` changed, as expected from the interface prediction. New files `test_express.py` and `test_interfaces.py` check the service and both adapters. Existing pricing, validation and lifecycle tests protect the older behavior.

The prediction was right about the interfaces and tests. It didn't separately list the README/spec updates; those were also needed for discoverability. The shared builder made CR-01 easier because the new fee rule was added once for both quote and place. Named constants made the new 1200-cent fee clear. The later money-variable rename didn't affect the original implementation work.

Final review added one combined case: a student express order at exactly $50, then reopen, cancel twice and place another order. It retains a 500-cent discount, a 1200-cent fee and a 5700-cent total. The full suite now has 50 passing tests.

6. AI engineering judgment

The three recommendations below were recorded during this final review, after CR-01. They aren't being passed off as earlier advice. The relevant interaction excerpt, decisions and evidence are at `docs/ai-interactions.md` in the public repo, and in the submitted AI-use record. No particular product name is needed to follow the technical discussion.

1 - Add a $50 express round-trip test: ACCEPT

Suggestion: combine the exact discount boundary with saved express ordering, reopening, cancellation and the next ID in one test.

Reason: the old suite checked those pieces separately, but didn't take the $50 express case through the whole saved-order flow. A wrong discount could otherwise become a permanent historical price even if simpler quote tests passed.

Action: add `test_exact_discount_threshold_round_trip` in `test_express.py`, commit `c5f1bb5`. It checks amounts 5000/500/1200/5700, the reopened snapshot, stock restoration, a second cancellation with unchanged save bytes and ORD-0002. It passes; the suite grows from 49 to 50 methods. Evidence: final log 07.

2 - Use dictionary lookup for every catalog search: MODIFY

Suggestion: replace scans with an index for faster product search.

Reason: an index is convenient for trusted internal lines. On raw incoming IDs, a list or dictionary can instead raise TypeError, while invalid service requests should raise ValueError. With five products there is no measured speed need.

Action: keep the modified approach already in the code: `_change_stock` indexes validated IDs, while public request validation keeps the small scan. Explicit type checks would be another safe option. `test_invalid_requests_do_not_modify_existing_state` passes for list/dict IDs, and duplicate-stock and lifecycle checks still pass. The decision in this review is to retain that split, not to claim a new change.

3 - Use round(subtotal * 0.1): REJECT

Suggestion: shorten the percentage calculation using floating-point round.

Reason: the spec requires half cents to round upward. For 5005 cents, the required discount is 501; the proposed expression returns 500 because Python rounds ties to even. `evidence/12-rounding-comparison.txt` records the actual comparison.

Action: keep the integer half-up formula. The 5005-cent fixture test passes with 501, and the downward-rounding mutation is caught. The rejected expression was run as a comparison, not inserted into the production calculation.

7. Final checks and files

The final code has 50 passing test methods on Windows, Python 3.12.10. The original four are still present. Seven selected wrong changes are all caught. These checks support the exercised behavior; they aren't exhaustive testing or a coverage percentage. Menu input, permission errors, EOF and Ctrl+C are simulated. Python 3.10 wasn't separately tested. Concurrent writers and crash recovery during saving remain outside the supplied functional spec.

Repository

https://github.com/McMerger/COMP3415-A2-Campus-Supply-Desk

The repo is public, on main. It includes the original starter, source, tests and running instructions. Commit messages were reworded earlier; file snapshots, authors and timestamps were preserved. The starter remains `82ed75e`. The refactor/test/CR sequence can still be inspected in history.

Useful code commits are `466ab2b` (shared helpers and constants), `1cd95b9` (defect fixes), `c3e234b` (CR tests), `e6abcca` (express), `319fa35` (money names) and `c5f1bb5` (the additional round-trip test). `22ccf9a` refreshes verification.

Run the application and checks

```
python -m supplydesk
python -m unittest discover -s tests -v
python scripts/verify.py
```

Logs 01–06 retain the original development results. Log 07 is the latest suite; 08 contains mutation output; 09 records source/test hashes. Logs 10/11 compare the later rename, and 12 checks the rejected rounding suggestion. The AI-use record and CR planning note are under `docs/` and supplied with the final files.

Submission and sources

This PDF answers Tasks 1–6. The other final files are the public repository link, testing evidence and AI-use record. Checkpoint A remains separate; its earlier submission was confirmed, but its contents weren't available for this revision. Nothing has been submitted to D2L by this work.

Sources: the supplied Assignment 2: Software Rescue Mission handout, starter README and FUNCTIONAL_SPEC.md, the supplied CR-01, and the recorded code/test results. The report has been checked against the actual handout rather than the earlier reconstructed guide. The CR planning evidence is the preserved chat note and test-before-implementation history, not a fabricated dated worksheet.